

Trilha DevOps + Cloud + SRE

Plano técnico completo para nível sênior

Tema	DevOps + Cloud + SRE
Nível-alvo	Avançado / Sênior
Foco	Entrega, operação, observabilidade, segurança e confiabilidade em produção
Formato	Plano de estudos técnico e progressivo

Documento formatado para estudo técnico, evolução profissional e consulta rápida.

Conteúdo base: arquivo anexado pelo usuário. Organização visual: capa, sumário, hierarquia de títulos, listas, checklists, blocos técnicos e paginação.

Sumário

Navegação estrutural do conteúdo principal.

1. Análise inicial do conteúdo	4
2. Explicação técnica de cada grande tópico	5
2.1 Linux, redes e base operacional	5
2.2 Docker e containers	7
2.3 CI/CD	9
2.4 Kubernetes profissional	11
2.5 Kubernetes avançado	13
2.6 Terraform e IaC	16
2.7 Observabilidade	18
2.8 SRE	20
2.9 DevSecOps e supply chain	22
3. Plano de estudos progressivo em 6 módulos	24
Módulo 1 — Fundamentos operacionais	24
Módulo 2 — Containers e aplicação prática	26
Módulo 3 — CI/CD e entrega contínua	28
Módulo 4 — Kubernetes, arquitetura e boas práticas	30
Módulo 5 — Produção, performance, segurança e	32
Módulo 6 — Domínio sênior e liderança técnica	35
4. Critérios de domínio técnico por grande área	37
5. Aplicação prática em sistemas reais	39
6. Cronograma sugerido de execução	42
7. Projetos recomendados para portfólio	45
Projeto 1 — Stack local DevOps	45

Projeto 2 — Pipeline CI/CD completo	46
Projeto 3 — Kubernetes Production-like	46
Projeto 4 — Observabilidade e SRE	46
Projeto 5 — Plataforma GitOps segura	47

8. Checklist de domínio técnico	47
--	-----------

9. Indicadores de prontidão para nível	49
---	-----------

1. Análise inicial do conteúdo

Tema central

A trilha aborda DevOps, Cloud, Kubernetes, Infraestrutura como Código, Observabilidade, SRE e DevSecOps, com foco em formar um profissional capaz de entregar, operar, proteger, monitorar e recuperar sistemas em produção.

O foco não é apenas “saber usar ferramentas”, mas desenvolver visão de engenharia de plataforma, confiabilidade, automação e operação real de sistemas distribuídos.

Kubernetes é hoje uma base central do ecossistema cloud native por automatizar implantação, escala e gerenciamento de aplicações containerizadas. OpenTelemetry se consolidou como padrão aberto para geração e exportação de telemetria, incluindo traces, métricas e logs. Terraform permanece como uma das principais ferramentas para gerenciar infraestrutura declarativa baseada em estado.

Objetivo principal da trilha

Formar um profissional capaz de:

- Entender sistemas Linux, redes e runtime de aplicações.
- Containerizar aplicações com segurança e eficiência.
- Criar pipelines CI/CD confiáveis, seguros e rastreáveis.
- Orquestrar workloads em Kubernetes.
- Provisionar infraestrutura reproduzível com IaC.
- Observar sistemas em produção com métricas, logs e traces.
- Definir SLOs, SLIs, error budgets e processos SRE.
- Proteger supply chain, imagens, secrets, artefatos e clusters.
- Liderar decisões técnicas envolvendo disponibilidade, custo, segurança e escalabilidade.

Competências técnicas a desenvolver

Ao longo da trilha, o estudante deve desenvolver competências em:

- Base operacional: Linux, processos, rede, DNS, TLS, troubleshooting.
- Containers: Dockerfile, Compose, imagens, segurança, SBOM, scanning.
- CI/CD: pipelines, quality gates, rollback, canary, release strategy.
- Kubernetes: workloads, networking, storage, autoscaling, RBAC, troubleshooting.
- Kubernetes avançado: service mesh, CRDs, operators, multi-cluster, policies.
- IaC: Terraform, módulos, state, remote state, drift, testes e governança.
- Observabilidade: logs, métricas, tracing, dashboards, alertas acionáveis.
- SRE: SLI, SLO, SLA, error budget, incidentes, runbooks, postmortems.
- DevSecOps: SAST, DAST, SCA, SBOM, image signing, policy as code, runtime security.

Classificação por nível

Básico

Linux, terminal, filesystem, permissões, HTTP, DNS, Docker básico, logs, comandos de diagnóstico, GitHub Actions simples.

Intermediário

Docker Compose, pipelines com build/test/deploy, Kubernetes básico, ConfigMaps, Secrets, Services, Ingress, Terraform básico, Prometheus/Grafana, logs estruturados.

Avançado

Kubernetes profissional, Helm, Kustomize, HPA, PDB, RBAC, troubleshooting, Terraform modules, remote state, observabilidade distribuída, tracing, alertas por SLO, pipelines seguros.

Sênior

Arquitetura de plataforma, multi-cluster, GitOps, service mesh, mTLS, canary, fault injection, policy as code, supply chain security, resposta a incidentes, capacity planning, DR, RPO/RTO, decisões de trade-off e liderança técnica.

2. Explicação técnica de cada grande tópico

2.1 Linux, redes e base operacional

O que é

É a base operacional onde aplicações, containers, agentes, bancos e serviços executam. Inclui filesystem, processos, permissões, systemd, logs, shell scripting, SSH, DNS, HTTP, TLS, TCP/IP, firewalls e troubleshooting.

Para que serve

Serve para diagnosticar problemas reais de produção: serviço fora do ar, porta bloqueada, certificado expirado, DNS incorreto, processo travado, consumo excessivo de CPU/memória, falha de conectividade ou lentidão de rede.

Por que é importante

Um profissional DevOps/SRE que não domina Linux e redes depende cegamente de ferramentas. Em produção, muitas falhas aparecem primeiro como sintomas de sistema operacional ou rede: timeout, conexão recusada, DNS resolvendo errado, TLS inválido, processo reiniciando ou falta de recursos.

Onde é usado em projetos reais

- Servidores de aplicação.
- Containers Linux.
- Nodes Kubernetes.
- VMs cloud.
- Bastion hosts.
- Load balancers.

- Gateways.
- Proxies reversos.
- Ambientes de CI/CD.

Problemas que resolve

- API fora do ar.
- Porta indisponível.
- DNS não resolve.
- Certificado TLS inválido.
- Processo em crash.
- Alto consumo de CPU/memória.
- Latência de rede.
- Firewall bloqueando tráfego.
- Serviço systemd falhando.

Erros comuns

- Decorar comandos sem entender o modelo TCP/IP.
- Confundir DNS, porta, rota e firewall.
- Não diferenciar erro de aplicação de erro de infraestrutura.
- Usar top/htop sem interpretar CPU steal, load average, memória e I/O.
- Ignorar logs do systemd com journalctl.
- Diagnosticar HTTP sem olhar TLS, DNS e rede.

Conhecimentos prévios

- Terminal Linux.
- Conceitos básicos de rede.
- HTTP.
- Noções de processos e portas.
- Permissões de arquivos.

Profundidade esperada para nível sênior

Um sênior deve conseguir responder rapidamente:

- O serviço está rodando?
- A porta está aberta?
- O DNS resolve corretamente?
- O certificado é válido?
- O tráfego chega ao destino?
- O firewall bloqueia algo?
- O processo está consumindo recurso demais?
- O problema está no cliente, rede, proxy, aplicação, banco ou cluster?

Comandos obrigatórios

```
curl
dig
nslookup
ss
netstat
tcpdump
traceroute
journalctl
top
htop
df
du
ps
systemctl
```

2.2 Docker e containers

O que é

Docker e containers permitem empacotar aplicação, runtime e dependências em uma unidade executável padronizada. A aplicação passa a rodar de forma previsível em ambiente local, CI, staging e produção.

Para que serve

Serve para padronizar execução, isolar dependências, reduzir divergência entre ambientes e facilitar deploy.

Por que é importante

A maior parte dos workloads modernos em Kubernetes roda como container. Um Dockerfile mal feito gera imagens grandes, inseguras, lentas de buildar e difíceis de manter.

A documentação oficial do Docker recomenda multi-stage builds para separar etapa de build e runtime, reduzindo tamanho da imagem final e mantendo apenas o necessário para execução. O Docker também documenta uso de SBOM para transparência da cadeia de software e análise de componentes da imagem.

Onde é usado em projetos reais

- Frontend Next.js/React.
- Backend Java/Spring Boot.
- APIs Node/Python.
- Bancos locais para desenvolvimento.
- Redis, Kafka, Postgres, MongoDB.
- Ambientes de CI.
- Jobs batch.
- Workers assíncronos.
- Stack de observabilidade local.

Problemas que resolve

- “Na minha máquina funciona.”
- Dependências inconsistentes.
- Runtime divergente.
- Builds manuais.
- Deploy frágil.
- Ambiente local complexo.
- Falta de padronização entre times.

Erros comuns

- Rodar container como root sem necessidade.
- Copiar arquivos demais para a imagem.
- Não usar .dockerignore.
- Não fixar versões de base image.
- Usar imagens muito grandes.
- Colocar secrets dentro do Dockerfile.
- Não definir healthcheck.
- Não otimizar cache de build.
- Não escanear vulnerabilidades.
- Não gerar SBOM.

Conhecimentos prévios

- Linux básico.
- Processo de build da aplicação.
- Variáveis de ambiente.
- Rede e portas.
- Conceito de imagem e container.

Profundidade sênior

Um sênior deve saber construir imagens:

- Pequenas.
- Reprodutíveis.
- Escaneáveis.
- Com usuário não-root.
- Com menor superfície de ataque.
- Com cache eficiente.
- Com SBOM.
- Sem secrets embutidos.
- Preparadas para Kubernetes.

Projeto prático

Containerizar uma stack completa:

- Frontend.
- Backend.
- Postgres.
- Redis.
- Kafka.
- Prometheus.
- Grafana.
- Loki.
- Jaeger ou Tempo.

Critério de conclusão

- docker compose up sobe tudo.
- Serviços se comunicam por rede interna.
- Backend expõe healthcheck.
- Logs são estruturados.
- Imagens são escaneadas.
- Dockerfiles usam multi-stage build.
- Containers rodam sem root quando possível.

2.3 CI/CD

O que é

CI/CD é o fluxo automatizado que leva código do commit até produção com build, testes, análise de qualidade, segurança, empacotamento, versionamento, deploy, rollback e rastreabilidade.

Para que serve

Serve para reduzir erro humano, acelerar entrega, padronizar releases, detectar problemas cedo e permitir recuperação rápida.

Por que é importante

Sem CI/CD maduro, a equipe depende de deploy manual, validação informal e rollback improvisado. Em ambientes críticos, isso aumenta risco de indisponibilidade, falha de segurança e perda de rastreabilidade.

As métricas DORA são usadas para avaliar desempenho de entrega de software, incluindo frequência de deploy, lead time de mudança, tempo de recuperação de falha e taxa de falha de mudança.

Onde é usado em projetos reais

- Build de backend Java.
- Build de frontend.
- Testes automatizados.

- Análise Sonar.
- SAST/SCA.
- Build de imagem Docker.
- Push para registry.
- Deploy em Kubernetes.
- Rollback automatizado.
- Geração de release notes.

Problemas que resolve

- Deploy manual.
- Falta de rastreabilidade.
- Releases quebradas.
- Testes esquecidos.
- Vulnerabilidades entrando em produção.
- Falta de versionamento.
- Rollback lento.
- Diferença entre staging e produção.

Erros comuns

- Pipeline lento demais.
- Não separar CI de CD.
- Fazer deploy sem quality gate.
- Ignorar testes.
- Não versionar artefatos.
- Não ter rollback.
- Usar secrets em texto puro.
- Não registrar evidências de build.
- Não medir DORA.
- Misturar deploy manual com automação sem controle.

Conhecimentos prévios

- Git.
- Build da linguagem usada.
- Testes.
- Docker.
- Registry.
- Noções de cloud/Kubernetes.

Profundidade sênior

Um sênior deve projetar pipelines com:

- Build rápido.

- Testes por camada.
- Quality gate.
- Security gate.
- Versionamento semântico.
- Artefatos imutáveis.
- Deploy auditável.
- Rollback seguro.
- Aprovação por ambiente.
- Estratégia blue/green ou canary.
- Métricas DORA.
- Rastreabilidade commit → build → imagem → release → deploy.

2.4 Kubernetes profissional

O que é

Kubernetes é uma plataforma de orquestração de containers que gerencia deploy, escala, rede, configuração, storage e disponibilidade de aplicações containerizadas. A documentação oficial define Kubernetes como um motor open source para automatizar implantação, escala e gerenciamento de aplicações containerizadas.

Para que serve

Serve para rodar aplicações distribuídas com resiliência, autoscaling, service discovery, configuração declarativa e gerenciamento padronizado de workloads.

Por que é importante

Kubernetes é uma das principais bases de plataformas modernas. Ele exige domínio de aplicação, infraestrutura, rede, segurança e operação.

Onde é usado em projetos reais

- APIs backend.
- Frontends.
- Workers.
- Jobs batch.
- Microserviços.
- Gateways.
- Event consumers.
- Serviços internos.
- Ambientes multi-tenant.
- Plataformas internas de engenharia.

Problemas que resolve

- Escala horizontal.
- Recuperação automática de pods.

- Deploy declarativo.
- Service discovery.
- Configuração por ambiente.
- Balanceamento interno.
- Segregação por namespace.
- Controle de acesso.
- Rollout e rollback.

Erros comuns

- Não definir requests e limits.
- Usar secrets de forma insegura.
- Não configurar readiness/liveness probes.
- Expor serviço errado.
- Não entender DNS interno.
- Usar latest em imagens.
- Não versionar manifests.
- Não configurar RBAC mínimo.
- Ignorar eventos do cluster.
- Debugar apenas logs e esquecer describe.

Conhecimentos prévios

- Docker.
- YAML.
- Redes.
- HTTP.
- Linux.
- CI/CD.
- Conceitos de aplicação stateless/stateful.

Profundidade sênior

Um sênior deve debugar:

- CrashLoopBackOff.
- ImagePullBackOff.
- Falha de readiness.
- Falha de DNS interno.
- Ingress mal configurado.
- OOMKilled.
- CPU throttling.
- PVC pendente.
- Service sem endpoint.
- RBAC negando acesso.

- Deployment travado.
- HPA não escalando.

Objetos obrigatórios

- Pod.
- Deployment.
- ReplicaSet.
- Service.
- Ingress.
- ConfigMap.
- Secret.
- Probe.
- Request/Limit.
- HPA/VPA.
- PDB.
- Job/CronJob.
- StatefulSet.
- Volume.
- StorageClass.
- Namespace.
- RBAC.
- Helm.
- Kustomize.
- Ingress é usado para expor serviços HTTP/HTTPS e pode fornecer roteamento baseado em host/path, terminação SSL e balanceamento para backends definidos no cluster.

2.5 Kubernetes avançado

O que é

É o domínio de extensibilidade, segurança, tráfego, multi-cluster e automação avançada em Kubernetes.

Inclui

- CRDs.
- Custom controllers.
- Operators.
- Admission webhooks.
- CNI.
- Calico.
- Cilium.
- NetworkPolicy.

- Service mesh.
- Istio.
- Linkerd.
- mTLS.
- Traffic shifting.
- Canary.
- Fault injection.
- Circuit breaking.
- Multi-cluster.
- ArgoCD multi-cluster.
- Fleet management.

Para que serve

Serve para construir plataformas robustas, seguras e escaláveis, com governança, automação e controle fino de tráfego.

Por que é importante

Em nível sênior, Kubernetes deixa de ser apenas “ambiente de deploy” e vira uma plataforma programável. O profissional precisa entender como estender, proteger e operar o cluster em escala.

Argo CD segue o modelo GitOps para Kubernetes: ele monitora o estado atual das aplicações no cluster e compara com o estado desejado versionado no Git, indicando divergência quando a aplicação está OutOfSync.

Onde é usado em projetos reais

- Plataforma interna de deploy.
- Multi-tenant Kubernetes.
- Ambientes regulados.
- Malha de microsserviços.
- Canary automatizado.
- Segurança zero trust interna.
- GitOps multi-ambiente.
- Políticas obrigatórias de cluster.
- Controle de tráfego entre serviços.

Problemas que resolve

- Drift entre Git e cluster.
- Deploy manual em múltiplos clusters.
- Comunicação insegura entre serviços.
- Ausência de governança.
- Falta de controle de tráfego.
- Deploy arriscado sem canary.

- Necessidade de políticas centralizadas.
- Gestão de rede avançada.
- Automação de operadores.

Erros comuns

- Adotar service mesh sem necessidade clara.
- Usar Istio sem maturidade operacional.
- Criar CRDs sem entender lifecycle.
- Não testar admission webhooks.
- Quebrar deploys com políticas restritivas demais.
- Não documentar exceções.
- Não medir overhead de sidecars.
- Usar multi-cluster sem estratégia de operação.
- Confundir GitOps com simples pipeline de deploy.

Conhecimentos prévios

- Kubernetes profissional.
- Redes.
- TLS/mTLS.
- CI/CD.
- Segurança.
- Observabilidade.
- GitOps.
- Arquitetura de microsserviços.

Profundidade sênior

Um sênior deve decidir:

- Quando usar service mesh e quando evitar.
- Quando aplicar canary no ingress, no mesh ou no CD.
- Como proteger tráfego leste-oeste com mTLS.
- Como estruturar clusters por ambiente, domínio ou criticidade.
- Como impor políticas sem travar produtividade.
- Como reduzir blast radius.
- Como operar multi-cluster com GitOps.
- Como diagnosticar falhas entre app, sidecar, proxy, CNI e DNS.

Projeto prático

Publicar o projeto DevCore em Kubernetes com:

- Helm.
- ArgoCD.
- Istio ou Linkerd.

- mTLS.
- Canary.
- Observabilidade.
- Network Policies.
- RBAC.
- Secrets externos.
- Dashboards.
- Alertas.
- Runbooks.

2.6 Terraform e IaC

O que é

Infraestrutura como Código é a prática de definir infraestrutura em arquivos versionados, revisáveis e reproduzíveis. Terraform usa state para mapear recursos reais à configuração declarativa e calcular mudanças necessárias antes da aplicação.

Para que serve

Serve para criar e gerenciar infraestrutura cloud de forma automatizada, auditável e padronizada.

Por que é importante

Sem IaC, ambientes são criados manualmente, sem rastreabilidade, com alto risco de divergência entre staging e produção. Em escala, isso gera drift, retrabalho, insegurança e dificuldade de recuperação.

Onde é usado em projetos reais

- VPC/VNet.
- Subnets.
- Security groups.
- Load balancers.
- Clusters Kubernetes.
- Bancos gerenciados.
- Redis.
- Filas.
- Buckets.
- IAM.
- DNS.
- Observabilidade cloud.
- Ambientes multi-account.

Problemas que resolve

- Infraestrutura manual.
- Ambientes inconsistentes.

- Falta de versionamento.
- Provisionamento lento.
- Dificuldade de auditoria.
- Baixa reprodutibilidade.
- Drift não detectado.
- Configuração insegura.

Erros comuns

- State local em projeto sério.
- Não usar locking.
- Misturar ambientes no mesmo state.
- Não versionar módulos.
- Fazer apply direto sem review.
- Usar variáveis sensíveis sem cuidado.
- Não lidar com import.
- Não detectar drift.
- Criar módulos genéricos demais.
- Não testar infraestrutura.

Conhecimentos prévios

- Cloud básica.
- Redes.
- Segurança IAM.
- Git.
- YAML/HCL.
- Noções de ambiente e arquitetura.

Profundidade sênior

Um sênior deve saber:

- Projetar módulos reutilizáveis.
- Separar state por domínio/ambiente.
- Usar remote state e locking.
- Controlar versionamento de módulos.
- Detectar e tratar drift.
- Criar workflow com plan/apply revisável.
- Usar Terratest quando necessário.
- Integrar Atlantis ou pipelines.
- Aplicar Sentinel/OPA.
- Trabalhar com multi-account.
- Justificar design de rede, IAM, custos e resiliência.

2.7 Observabilidade

O que é

Observabilidade é a capacidade de entender o estado interno de um sistema analisando seus sinais externos: logs, métricas e traces. OpenTelemetry define instrumentação para emitir, coletar e exportar telemetria, incluindo traces, métricas e logs.

Para que serve

Serve para responder perguntas críticas:

- O sistema está saudável?
- Onde está o gargalo?
- Qual usuário foi impactado?
- Qual endpoint está lento?
- Qual dependência falhou?
- O erro começou após qual deploy?
- O problema é aplicação, banco, rede, cache ou mensageria?

Por que é importante

Sem observabilidade, o time opera no escuro. Logs soltos não bastam. Métricas sem contexto não bastam. Traces sem correlação não bastam. O valor real surge quando logs, métricas e traces são correlacionados com contexto, como correlationId, traceId, usuário, tenant, versão e ambiente.

Onde é usado em projetos reais

- APIs.
- Microserviços.
- Consumers Kafka.
- Jobs.
- Gateways.
- Bancos.
- Cache.
- Filas.
- Kubernetes.
- Pipelines.
- Incidentes.
- Capacity planning.

Problemas que resolve

- Incidentes difíceis de diagnosticar.
- Falta de visibilidade de latência.
- Erros intermitentes.
- Gargalos invisíveis.
- Alertas ruidosos.
- Dificuldade de provar impacto.

- Falta de rastreabilidade ponta a ponta.

Erros comuns

- Logar demais sem estrutura.
- Logar dados sensíveis.
- Criar dashboards bonitos sem ação.
- Alertar por CPU sem relação com usuário.
- Não usar traceId/correlationId.
- Não instrumentar dependências externas.
- Confundir monitoramento com observabilidade.
- Ter alerta sem runbook.
- Alertar sintomas irrelevantes.
- Não medir SLO.

Conhecimentos prévios

- HTTP.
- APIs.
- Logs.
- Métricas.
- Banco de dados.
- Mensageria.
- Kubernetes.
- Noções de performance.

Profundidade sênior

Um sênior deve projetar observabilidade orientada a usuário e negócio:

- RED metrics: rate, errors, duration.
- USE metrics: utilization, saturation, errors.
- Golden signals: latência, tráfego, erros e saturação.
- Logs estruturados.
- Tracing distribuído.
- Métricas de negócio.
- Dashboards por serviço.
- Dashboards executivos.
- Alertas acionáveis.
- Alertas por SLO.
- Correlação entre deploy e incidente.

Stack sugerida

- Prometheus.
- Grafana.

- Loki.
- Tempo ou Jaeger.
- OpenTelemetry Collector.
- Alertmanager.

2.8 SRE

O que é

SRE, Site Reliability Engineering, é a disciplina que aplica engenharia de software à operação de sistemas. O foco é confiabilidade mensurável, automação, redução de toil, resposta a incidentes e equilíbrio entre velocidade de entrega e estabilidade.

O livro de SRE do Google define SLO como um objetivo de nível de serviço medido por um SLI; o SRE Workbook também apresenta alertas multi-window multi-burn-rate para reduzir falsos positivos e alertar quando o error budget continua sendo consumido.

Para que serve

Serve para operar sistemas com metas claras de confiabilidade, em vez de decisões subjetivas como “o sistema parece bom”.

Por que é importante

SRE conecta engenharia, operação e negócio. Um sistema não precisa ser “100% disponível” em todos os casos; ele precisa atingir o nível de confiabilidade adequado ao produto, custo, risco e expectativa do usuário.

Onde é usado em projetos reais

- APIs críticas.
- Plataformas financeiras.
- E-commerce.
- Telecom.
- Sistemas de pedido.
- Sistemas com Kafka.
- Ambientes cloud.
- Plataformas SaaS.
- Times com on-call.
- Organizações com alta frequência de deploy.

Problemas que resolve

- Incidentes recorrentes.
- Falta de prioridade para confiabilidade.
- Alertas excessivos.
- Deploys arriscados.
- Ausência de runbooks.
- Postmortems culpabilizadores.
- Falta de capacity planning.

- Recuperação lenta.
- Baixa maturidade operacional.

Erros comuns

- Criar SLO sem SLI confiável.
- Definir SLO irreal.
- Não usar error budget para decisão.
- Fazer postmortem com busca por culpado.
- Não criar action items.
- Não simular incidentes.
- Não treinar on-call.
- Confundir SLA contratual com SLO interno.
- Alertar tudo.
- Não reduzir toil.

Conhecimentos prévios

- Observabilidade.
- Deploy.
- Kubernetes.
- Cloud.
- Arquitetura.
- Testes.
- Incidentes.
- Comunicação técnica.

Profundidade sênior

Um sênior deve conseguir:

- Definir SLIs por jornada crítica.
- Definir SLO por criticidade.
- Calcular error budget.
- Usar burn rate.
- Criar alertas multi-window.
- Liderar incident response.
- Escrever postmortem blameless.
- Criar runbooks.
- Planejar capacidade.
- Conduzir GameDays.
- Definir RPO/RTO.
- Desenhar estratégia de DR.
- Reduzir toil com automação.
- Decidir quando pausar feature para restaurar confiabilidade.

Projeto prático

Simular incidentes:

- Banco fora.
- Kafka indisponível.
- Latência alta.
- Erro 500 em massa.
- Deploy quebrado.
- Perda de cache.
- DNS quebrado.
- Certificado expirado.
- Pod com OOMKilled.
- Ingress retornando 502.

Entregáveis

- Timeline do incidente.
- Impacto.
- SLI afetado.
- SLO violado ou não.
- Causa raiz técnica.
- Mitigação.
- Correção definitiva.
- Ações preventivas.
- Runbook.
- Postmortem blameless.

2.9 DevSecOps e supply chain

O que é

DevSecOps integra segurança ao ciclo de desenvolvimento, build, entrega e operação. Supply chain security protege dependências, imagens, artefatos, pipelines, assinaturas, SBOMs e políticas.

SLSA é um framework de segurança para cadeia de suprimentos de software, com controles para prevenir adulteração, melhorar integridade e proteger pacotes e infraestrutura. OPA é um mecanismo de políticas usado para aplicar regras de segurança, auditoria e governança em diferentes camadas da stack.

Para que serve

Serve para impedir que vulnerabilidades, secrets, dependências comprometidas ou artefatos adulterados cheguem a produção.

Por que é importante

A segurança moderna não pode depender apenas de revisão manual no final. Ela precisa estar no pipeline, na imagem, no registry, no cluster, no runtime e nas políticas organizacionais.

Onde é usado em projetos reais

- Pipeline CI/CD.
- Pull requests.
- Registry de imagens.
- Kubernetes admission control.
- Runtime security.
- Gestão de secrets.
- Dependabot.
- SBOM.
- Assinatura de imagem.
- Política de deploy.
- Auditoria.

Problemas que resolve

- Secrets vazados.
- Dependências vulneráveis.
- Imagens com CVEs críticas.
- Artefatos adulterados.
- Deploy de imagem não assinada.
- Containers privilegiados.
- Workloads rodando como root.
- Falta de rastreabilidade.
- Políticas inconsistentes.

Erros comuns

- Rodar scanner sem bloquear nada.
- Bloquear tudo sem estratégia de exceção.
- Não classificar severidade.
- Não gerar SBOM.
- Não assinar imagens.
- Não proteger secrets.
- Não usar least privilege.
- Não auditar permissões.
- Não ter política de exceção.
- Tratar segurança como etapa final.

Conhecimentos prévios

- CI/CD.
- Docker.
- Kubernetes.
- Segurança básica.

- IAM/RBAC.
- Dependências.
- Build de aplicação.

Profundidade sênior

Um sênior deve proteger:

- Pipeline.
- Dependências.
- Artefatos.
- Imagens.
- Registry.
- Secrets.
- Cluster.
- RBAC.
- Policies.
- Runtime.

Ferramentas e práticas

- SAST.
- DAST.
- SCA.
- SBOM.
- Trivy.
- Grype.
- Syft.
- Cosign.
- Sigstore.
- SLSA.
- Dependabot.
- Secret scanning.
- Image signing.
- OPA/Gatekeeper.
- Falco.

3. Plano de estudos progressivo em 6 módulos

Módulo 1 — Fundamentos operacionais

Objetivo

Construir base sólida em Linux, redes, HTTP, DNS, TLS e troubleshooting.

Conteúdos

- Linux filesystem.
- Processos.
- Permissões.
- systemd.
- Logs.
- Shell scripting.
- SSH.
- DNS.
- HTTP.
- TLS.
- TCP/IP.
- Load balancing.
- Firewalls.
- Ferramentas de diagnóstico.

Conceitos obrigatórios

- Processo vs thread.
- Porta vs socket.
- DNS A, CNAME, TTL.
- HTTP status codes.
- TLS handshake.
- Certificado.
- Load balancer L4 vs L7.
- Firewall inbound/outbound.
- Logs de sistema.
- Exit code.
- Variáveis de ambiente.

Práticas recomendadas

- Subir uma API local.
- Expor em porta específica.
- Diagnosticar com curl.
- Ver processo com ps.
- Ver porta com ss.
- Resolver DNS com dig.
- Capturar pacote com tcpdump.
- Analisar serviço com journalctl.

Exercícios técnicos

- Diagnosticar serviço que não sobe.

- Corrigir permissão de arquivo.
- Identificar porta ocupada.
- Simular DNS inválido.
- Simular certificado expirado em ambiente local.
- Criar script de healthcheck.

Mini projeto

Criar uma VM Linux ou ambiente WSL com:

- Nginx.
- API simples.
- systemd service.
- Logs.
- Script de diagnóstico.
- Checklist de troubleshooting.

Critérios de conclusão

Você deve conseguir explicar e diagnosticar:

- Processo morto.
- Porta fechada.
- DNS errado.
- TLS inválido.
- Firewall bloqueando.
- Alto consumo de CPU.
- Falha de permissão.
- Serviço systemd quebrado.

Nível esperado

Intermediário operacional.

Módulo 2 — Containers e aplicação prática

Objetivo

Aprender a empacotar aplicações com Docker de forma segura, eficiente e reproduzível.

Conteúdos

- Dockerfile.
- Multi-stage build.
- Distless images.
- Docker Compose.
- Volumes.
- Networks.
- Healthcheck.

- Build cache.
- Layer optimization.
- Non-root user.
- Secrets.
- Image scanning.
- SBOM.

Conceitos obrigatórios

- Imagem vs container.
- Layer.
- Build context.
- Cache invalidation.
- Runtime image.
- Entrypoint vs CMD.
- Network bridge.
- Volume nomeado.
- Healthcheck.
- Imutabilidade de imagem.
- SBOM.

Práticas recomendadas

- Usar .dockerignore.
- Fixar versões.
- Separar build e runtime.
- Rodar como non-root.
- Remover dependências desnecessárias.
- Escanear imagem.
- Gerar SBOM.
- Nunca embutir secrets.

Exercícios técnicos

- Criar Dockerfile de backend.
- Criar Dockerfile de frontend.
- Otimizar tamanho da imagem.
- Adicionar healthcheck.
- Rodar Postgres, Redis e Kafka com Compose.
- Escanear imagem com Trivy/Grype.
- Gerar SBOM com Syft.

Mini projeto

Stack local completa:

- Frontend.
- Backend.
- Postgres.
- Redis.
- Kafka.
- Prometheus.
- Grafana.
- Loki.
- Jaeger/Tempo.

Critérios de conclusão

- Stack sobe com um comando.
- Serviços se comunicam por nome.
- Imagens otimizadas.
- Healthchecks funcionando.
- Imagens escaneadas.
- SBOM gerado.
- Secrets não estão no Dockerfile.

Nível esperado

Intermediário forte em containers.

Módulo 3 — CI/CD e entrega contínua

Objetivo

Construir pipelines seguros, rápidos, rastreáveis e preparados para rollback.

Conteúdos

- GitHub Actions.
- Jenkins.
- Build.
- Test.
- Quality gates.
- Security gates.
- Artifact registry.
- Semantic versioning.
- Release notes.
- Deploy.
- Rollback.
- Blue/green.
- Canary.
- Feature flags.

- DORA metrics.

Conceitos obrigatórios

- CI vs CD.
- Pipeline stage.
- Artifact.
- Registry.
- Quality gate.
- Security gate.
- Versionamento semântico.
- Rollback.
- Promotion entre ambientes.
- Estratégia de release.
- Lead time.
- Change failure rate.
- Recovery time.

Práticas recomendadas

- Pipeline por pull request.
- Testes automatizados.
- Build imutável.
- Imagem versionada por commit SHA.
- Scan de dependências.
- Scan de imagem.
- Deploy via GitOps ou manifesto versionado.
- Rollback testado.
- Release notes automatizadas.

Exercícios técnicos

- Criar pipeline de PR.
- Criar pipeline de release.
- Publicar imagem no registry.
- Bloquear merge com teste falhando.
- Bloquear deploy com vulnerabilidade crítica.
- Criar rollback.
- Medir DORA manualmente.

Mini projeto

Pipeline completo para aplicação containerizada:

- Build.
- Unit tests.

- Integration tests.
- Sonar/quality gate.
- SCA.
- Docker build.
- Image scanning.
- Push registry.
- Deploy staging.
- Aprovação.
- Deploy produção.
- Rollback.

Critérios de conclusão

- Pipeline é reproduzível.
- Build gera artefato rastreável.
- Deploy não depende de comando manual local.
- Há rollback.
- Há quality/security gates.
- Métricas básicas de entrega são coletadas.

Nível esperado

Avançado inicial em delivery.

Módulo 4 — Kubernetes, arquitetura e boas práticas

Objetivo

Publicar e operar aplicações em Kubernetes com segurança, resiliência e padronização.

Conteúdos

- Pods.
- Deployments.
- ReplicaSets.
- Services.
- Ingress.
- ConfigMaps.
- Secrets.
- Probes.
- Requests.
- Limits.
- HPA.
- VPA.
- PDB.
- Jobs.

- CronJobs.
- StatefulSets.
- Volumes.
- StorageClass.
- Namespaces.
- RBAC.
- Helm.
- Kustomize.
- Troubleshooting.

Conceitos obrigatórios

- Desired state.
- Reconciliation loop.
- Service discovery.
- ClusterIP, NodePort, LoadBalancer.
- Readiness vs liveness.
- Request vs limit.
- Rolling update.
- OOMKilled.
- CPU throttling.
- RBAC least privilege.
- Helm chart.
- Overlay com Kustomize.

Práticas recomendadas

- Nunca usar latest.
- Definir requests/limits.
- Definir readiness/liveness.
- Separar namespace por ambiente/domínio.
- Aplicar RBAC mínimo.
- Criar Helm charts reutilizáveis.
- Versionar manifests.
- Usar GitOps.
- Criar dashboards por workload.
- Criar runbooks de incidentes comuns.

Exercícios técnicos

- Publicar backend.
- Publicar frontend.
- Criar Service.
- Criar Ingress.

- Configurar secrets.
- Adicionar probes.
- Configurar HPA.
- Simular CrashLoopBackOff.
- Simular ImagePullBackOff.
- Simular erro de readiness.
- Corrigir OOMKilled.

Mini projeto

Publicar a aplicação completa em Kubernetes:

- Frontend.
- Backend.
- Postgres ou banco gerenciado simulado.
- Redis.
- Kafka.
- Ingress.
- HPA.
- Secrets.
- ConfigMaps.
- Observabilidade básica.
- Helm chart.

Critérios de conclusão

- Aplicação acessível via Ingress.
- Rollout e rollback funcionando.
- Probes funcionando.
- Requests/limits definidos.
- HPA configurado.
- RBAC mínimo.
- Debug de falhas comuns documentado.

Nível esperado

Avançado em Kubernetes profissional.

Módulo 5 — Produção, performance, segurança e

escalabilidade

Objetivo

Preparar sistemas para produção com observabilidade, SRE, segurança, IaC e confiabilidade.

Conteúdos

- Terraform modules.

- Remote state.
- State locking.
- Workspaces.
- Drift detection.
- Terratest.
- Atlantis.
- OPA/Sentinel.
- OpenTelemetry.
- Prometheus.
- Grafana.
- Loki.
- Tempo/Jaeger.
- Alertmanager.
- SLIs.
- SLOs.
- Error budget.
- Burn rate.
- Incident response.
- Runbooks.
- Postmortems.
- Capacity planning.
- Load testing.
- DR.
- RPO/RTO.
- DevSecOps.
- SBOM.
- Image signing.
- Policy as code.
- Runtime security.

Conceitos obrigatórios

- Infraestrutura declarativa.
- State.
- Drift.
- Observabilidade distribuída.
- Trace ID.
- Correlation ID.
- Golden signals.
- RED/USE.

- SLI/SLO/SLA.
- Error budget.
- RPO/RTO.
- SAST/DAST/SCA.
- SBOM.
- Supply chain security.
- Policy as code.

Práticas recomendadas

- Instrumentar aplicações.
- Criar dashboards orientados a SLO.
- Alertar apenas o que exige ação.
- Criar runbooks.
- Fazer postmortem blameless.
- Testar restauração.
- Escanear dependências.
- Assinar imagens.
- Bloquear deploy inseguro.
- Aplicar policies no cluster.
- Simular incidentes.

Exercícios técnicos

- Criar infraestrutura com Terraform.
- Detectar drift.
- Criar dashboard RED.
- Criar tracing ponta a ponta.
- Criar alerta por SLO.
- Simular indisponibilidade do banco.
- Simular erro 500 em massa.
- Criar postmortem.
- Assinar imagem com Cosign.
- Bloquear container root com policy.

Mini projeto

Ambiente de produção simulado:

- Infra com Terraform.
- Deploy com ArgoCD.
- Kubernetes com Helm.
- Observabilidade completa.
- SLO por endpoint crítico.
- Alertas acionáveis.

- Segurança no pipeline.
- Políticas no cluster.
- Incidentes simulados.
- Runbooks e postmortems.

Critérios de conclusão

- Infra é reproduzível.
- Deploy é GitOps.
- Sistema é observável.
- Alertas são úteis.
- Incidentes têm processo.
- Pipeline bloqueia riscos críticos.
- Há estratégia de rollback.
- Há RPO/RTO definidos.

Nível esperado

Avançado forte, próximo de atuação sênior.

Módulo 6 — Domínio sênior e liderança técnica

Objetivo

Desenvolver capacidade de decisão, arquitetura, revisão técnica, governança e liderança em ambientes de produção.

Conteúdos

- Trade-offs arquiteturais.
- Plataforma interna.
- Multi-cluster.
- Service mesh.
- mTLS.
- Canary.
- Fault injection.
- Circuit breaking.
- Chaos engineering.
- GameDays.
- Fleet management.
- Governança cloud.
- FinOps básico.
- Segurança por padrão.
- Revisão de arquitetura.
- Estratégia de evolução.
- Documentação técnica.

- Mentoria técnica.

Conceitos obrigatórios

- Blast radius.
- Failure domain.
- High availability.
- Disaster recovery.
- Zero trust interno.
- GitOps em escala.
- Platform engineering.
- Custo vs confiabilidade.
- Segurança vs produtividade.
- Complexidade operacional.
- Manutenibilidade.
- Evolução incremental.

Práticas recomendadas

- Escrever ADRs.
- Conduzir design review.
- Criar padrões reutilizáveis.
- Definir golden paths.
- Documentar runbooks.
- Medir confiabilidade.
- Reduzir toil.
- Automatizar tarefas repetitivas.
- Guiar times no uso da plataforma.
- Revisar risco antes de produção.

Exercícios técnicos

- Escrever ADR sobre service mesh.
- Escrever ADR sobre multi-cluster.
- Comparar blue/green vs canary.
- Definir SLO para API crítica.
- Fazer revisão de pipeline.
- Fazer revisão de Terraform.
- Fazer revisão de manifests Kubernetes.
- Planejar DR.
- Conduzir GameDay.
- Criar roadmap de maturidade DevOps/SRE.

Mini projeto

Criar uma plataforma interna simplificada:

- Template de serviço.
- Dockerfile padrão.
- Helm chart padrão.
- Pipeline padrão.
- Observabilidade padrão.
- Políticas padrão.
- Runbook padrão.
- Dashboard padrão.
- ArgoCD app padrão.
- Documentação para times.

Critérios de conclusão

Você deve conseguir:

- Projetar solução.
- Defender trade-offs.
- Revisar arquitetura.
- Diagnosticar produção.
- Liderar incidente.
- Criar padrões.
- Reduzir risco.
- Mentorar outros devs.
- Medir impacto técnico.
- Justificar custo, segurança e confiabilidade.

Nível esperado

Sênior em DevOps/SRE/Cloud Platform.

4. Critérios de domínio técnico por grande área

Base operacional

Iniciante

I Entende comandos básicos, arquivos, permissões, processos e portas.

Intermediário

I Diagnostica serviço fora do ar, DNS, HTTP, logs e consumo de recurso.

Avançado

Investiga problemas entre aplicação, rede, sistema operacional e infraestrutura.

Sênior

I Define processo de troubleshooting, cria runbooks, reduz MTTR e ensina o time a diagnosticar produção.

Containers

Iniciante

Sabe criar e rodar containers.

Intermediário

I Cria Dockerfiles funcionais e usa Docker Compose.

Avançado

Otimiza imagem, cache, segurança, usuário não-root, healthchecks e scanning.

Sênior

I Define padrões corporativos de imagem, supply chain, SBOM, registry, segurança e runtime.

CI/CD

Iniciante

I Entende build e testes automatizados.

Intermediário

I Cria pipeline com build, teste e deploy.

Avançado

Adiciona quality gates, security gates, versionamento, rollback e artefatos.

Sênior

Desenha estratégia de entrega com rastreabilidade, DORA, compliance, canary, rollback e governança.

Kubernetes

Iniciante

I Entende Pod, Deployment e Service.

Intermediário

I Publica aplicação com Ingress, ConfigMap, Secret e probes.

Avançado

Domina HPA, requests/limits, RBAC, Helm, troubleshooting e storage.

Sênior

Projeta plataforma Kubernetes segura, resiliente, multi-ambiente, observável e governada.

Observabilidade e SRE

Iniciante

I Entende logs, métricas e dashboards.

Intermediário

I Cria dashboards e alertas básicos.

Avançado

Instrumenta traces, métricas, logs estruturados, SLOs e runbooks.

Sênior

I Define confiabilidade por jornada de usuário, error budget, burn rate, resposta a incidentes, postmortems e redução de toil.

DevSecOps

Iniciante

I Entende vulnerabilidades e secrets.

Intermediário

I Usa scanners em dependências e imagens.

Avançado

I Gera SBOM, assina imagens, aplica policies e protege pipeline.

Sênior

Projeta cadeia de entrega segura de ponta a ponta, com governança, exceções, auditoria, runtime security e decisões baseadas em risco.

5. Aplicação prática em sistemas reais

APIs

Aplicar

- Dockerfile otimizado.
- Pipeline com testes.
- Deploy em Kubernetes.
- Readiness/liveness probes.
- Logs estruturados.
- Trace ID.
- Métricas RED.
- SLO por endpoint crítico.
- Canary para novas versões.
- Exemplo: API de pedidos com endpoint /orders, SLO de 99,9% de sucesso e latência p95 abaixo de 300

- ms.

Banco de dados

Aplicar

- Healthcheck.
- Backup.
- Restore testado.
- Métricas de conexões.
- Latência de queries.
- Capacidade de storage.
- RPO/RTO.
- Runbook de indisponibilidade.
- Exemplo: Postgres com alerta de saturação de conexões e runbook para failover.

Microsserviços

Aplicar

- Service discovery.
- Tracing distribuído.
- Circuit breaker.
- Retry com backoff.
- Timeout.
- Bulkhead.
- mTLS quando necessário.
- SLO por jornada, não apenas por serviço isolado.
- Exemplo: pedido → pagamento → estoque → notificação, com trace único por requisição.

Mensageria

Aplicar

- Métricas de consumer lag.
- Dead letter queue.
- Retry controlado.
- Idempotência.
- Observabilidade por evento.
- Runbook de Kafka indisponível.
- Teste de backpressure.
- Exemplo: consumer Kafka com alerta quando lag cresce acima do limite por mais de X minutos.

Segurança

Aplicar

- Secret scanning.

- SCA.
- SAST.
- Image scanning.
- SBOM.
- Cosign.
- OPA/Gatekeeper.
- RBAC mínimo.
- NetworkPolicy.
- Falco.
- Exemplo: pipeline bloqueia imagem com CVE crítica explorável e cluster bloqueia pod privilegiado.

Observabilidade

Aplicar

- OpenTelemetry.
- Prometheus.
- Grafana.
- Loki.
- Tempo/Jaeger.
- Alertmanager.
- Dashboards por serviço.
- Alertas por SLO.
- Correlation ID.
- Exemplo: erro 500 em massa é identificado por dashboard, trace e logs correlacionados.

Testes

Aplicar

- Unit tests.
- Integration tests.
- Contract tests.
- Load tests.
- Chaos tests controlados.
- Smoke tests pós-deploy.
- Teste de rollback.
- Exemplo: pipeline só promove para produção após smoke test no ambiente staging.

Deploy

Aplicar

- Blue/green.
- Canary.

- Feature flags.
- Rollback.
- ArgoCD.
- Helm.
- Release notes.
- Versionamento semântico.
- Exemplo: liberar nova versão para 10% do tráfego, validar SLO e expandir gradualmente.

Arquitetura

Aplicar

- ADRs.
- Trade-offs.
- Failure domains.
- Blast radius.
- Resiliência.
- Escalabilidade horizontal.
- Segurança por padrão.
- Observabilidade desde o design.
- Exemplo: decidir entre monólito modular, microsserviços ou arquitetura orientada a eventos
- com base em autonomia, escala, complexidade e custo operacional.

6. Cronograma sugerido de execução

Duração recomendada

Para uma formação forte e realista: 28 semanas, considerando estudo consistente, prática e projetos.

Distribuição semanal

Semanas 1 a 4 — Fundamentos operacionais

Prioridade

- Linux.
- Processos.
- Permissões.
- systemd.
- Logs.
- Redes.
- DNS.
- HTTP/TLS.
- Troubleshooting.

Checkpoint

- Diagnosticar 10 falhas simuladas em ambiente Linux.

Semanas 5 a 8 — Docker e containers

Prioridade

- Dockerfile.
- Multi-stage build.
- Compose.
- Networks.
- Volumes.
- Healthcheck.
- Non-root.
- Scanning.
- SBOM.

Checkpoint

- Stack completa local containerizada.

Semanas 9 a 12 — CI/CD

Prioridade

- GitHub Actions.
- Jenkins.
- Build/test.
- Quality gate.
- Security gate.
- Docker build.
- Registry.
- Deploy.
- Rollback.
- DORA.

Checkpoint

- Pipeline completo com build, testes, scan, imagem e deploy.

Semanas 13 a 18 — Kubernetes profissional

Prioridade

- Pods.
- Deployments.
- Services.
- Ingress.
- ConfigMaps.

- Secrets.
- Probes.
- Requests/limits.
- HPA.
- RBAC.
- Helm.
- Kustomize.
- Troubleshooting.

Checkpoint

- Aplicação publicada em Kubernetes com Helm, Ingress, HPA e RBAC.

Semanas 19 a 22 — Terraform, observabilidade e produção

Prioridade

- Terraform modules.
- Remote state.
- Locking.
- Drift.
- OpenTelemetry.
- Prometheus.
- Grafana.
- Loki.
- Tempo/Jaeger.
- Alertmanager.
- Dashboards.
- Alertas.

Checkpoint

- Infra reproduzível com Terraform e aplicação observável ponta a ponta.

Semanas 23 a 25 — SRE

Prioridade

- SLIs.
- SLOs.
- Error budget.
- Burn rate.
- Incident response.
- Postmortem.
- Runbooks.
- Capacity planning.
- Load testing.

- RPO/RTO.
- GameDays.

Checkpoint

- Simular 5 incidentes e produzir postmortems completos.

Semanas 26 a 28 — DevSecOps e domínio sênior

Prioridade

- SAST.
- DAST.
- SCA.
- SBOM.
- Cosign.
- Sigstore.
- OPA/Gatekeeper.
- Falco.
- NetworkPolicy.
- Supply chain.
- Design review.
- ADRs.

Checkpoint

- Pipeline e cluster com políticas de segurança, imagem assinada, SBOM e bloqueios básicos.

7. Projetos recomendados para portfólio

Projeto 1 — Stack local DevOps

Aplicação com:

- Frontend.
- Backend.
- Postgres.
- Redis.
- Kafka.
- Observabilidade.
- Docker Compose.
- Objetivo: provar domínio de containers e ambiente local.

Projeto 2 — Pipeline CI/CD completo

Inclui

- Build.
- Testes.
- Quality gate.
- Security gate.
- Docker build.
- Registry.
- Versionamento.
- Release notes.
- Deploy.
- Rollback.
- Objetivo: provar domínio de entrega contínua.

Projeto 3 — Kubernetes Production-like

Inclui

- Helm.
- Ingress.
- ConfigMaps.
- Secrets.
- Probes.
- HPA.
- PDB.
- RBAC.
- NetworkPolicy.
- Troubleshooting documentado.
- Objetivo: provar domínio de Kubernetes profissional.

Projeto 4 — Observabilidade e SRE

Inclui

- OpenTelemetry.
- Prometheus.
- Grafana.
- Loki.
- Tempo/Jaeger.
- Alertmanager.
- SLOs.
- Runbooks.

- Postmortems.
- Objetivo: provar domínio de operação e confiabilidade.

Projeto 5 — Plataforma GitOps segura

Inclui

- ArgoCD.
- Helm.
- Terraform.
- OPA/Gatekeeper.
- Cosign.
- SBOM.
- Trivy/Grype.
- Falco.
- Canary.
- mTLS opcional.
- Objetivo: provar domínio sênior de plataforma, segurança e operação.

8. Checklist de domínio técnico

Linux e redes

- I Diagnostica processo, porta, DNS, TLS e firewall.
- I Usa curl, dig, ss, tcpdump, journalctl.
- I Entende HTTP, TCP/IP, load balancing e logs.
- I Cria runbooks de troubleshooting.

Docker

- I Cria Dockerfile multi-stage.
- I Usa non-root user.
- Otimiza cache e layers.
- I Usa Compose com networks e volumes.
- Escaneia imagem.
- I Gera SBOM.
- I Não embute secrets.

CI/CD

- I Cria pipeline completo.
- I Usa quality gates.
- I Usa security gates.
- I Publica artefatos versionados.
- I Implementa rollback.

- I Mede DORA.
- I Gera release notes.

Kubernetes

- I Publica aplicações com Deployment, Service e Ingress.
- I Usa ConfigMaps e Secrets.
- I Define probes.
- I Define requests e limits.
- I Configura HPA.
- I Usa RBAC.
- I Usa Helm/Kustomize.
- I Debuga falhas reais.

Terraform

- I Usa módulos.
- I Usa remote state.
- I Usa locking.
- Detecta drift.
- Versiona módulos.
- I Usa plan/apply revisável.
- I Entende import.
- I Aplica policies.

Observabilidade

- I Cria logs estruturados.
- I Usa correlationId/traceld.
- I Cria métricas RED/USE.
- I Usa tracing distribuído.
- I Cria dashboards úteis.
- I Cria alertas acionáveis.
- Responde impacto por usuário/jornada.

SRE

- I Define SLIs.
- I Define SLOs.
- Calcula error budget.
- I Usa burn rate.
- I Cria runbooks.
- I Faz postmortem blameless.
- Planeja capacidade.
- I Define RPO/RTO.

- Conduz GameDays.

DevSecOps

- I Usa SAST.
- I Usa DAST.
- I Usa SCA.
- I Gera SBOM.
- I Assina imagens.
- I Faz secret scanning.
- I Aplica policy as code.
- I Usa runtime security.
- I Protege cluster, pipeline e artefatos.

9. Indicadores de prontidão para nível

avançado/sênior

Você está pronto para atuar em nível avançado/sênior nessa trilha quando conseguir:

- Diagnosticar incidentes de produção sem depender apenas de logs.
- Explicar se uma falha está na aplicação, rede, DNS, cluster, banco ou pipeline.
- Criar um Dockerfile seguro, pequeno e reproduzível.
- Criar pipeline com testes, qualidade, segurança, versionamento, deploy e rollback.
- Publicar aplicação em Kubernetes com Helm, HPA, probes, RBAC e Ingress.
- Debugar CrashLoopBackOff, ImagePullBackOff, OOMKilled, readiness failure e DNS interno.
- Criar infraestrutura com Terraform usando state remoto, locking e módulos.
- Instrumentar uma aplicação com logs, métricas e traces.
- Criar dashboards que ajudam a tomar decisão.
- Criar alertas que realmente exigem ação.
- Definir SLOs e error budgets por jornada crítica.
- Liderar resposta a incidente com comunicação técnica clara.
- Escrever postmortem sem culpabilização.
- Proteger pipeline, imagem, registry, secrets e cluster.
- Justificar trade-offs entre custo, segurança, disponibilidade, complexidade e velocidade de entrega.
- Criar padrões reutilizáveis para outros times.
- Revisar arquitetura de produção com foco em risco real.

Resumo final da trilha

A trilha DevOps + Cloud + SRE forma um profissional capaz de sair do nível operacional básico e chegar a uma atuação de engenharia de plataforma e confiabilidade.

O caminho começa com Linux e redes, evolui para containers, CI/CD e Kubernetes, aprofunda em Terraform, observabilidade e SRE, e finaliza com DevSecOps, supply chain security, GitOps, service mesh, incident response e liderança técnica.

O diferencial sênior não é conhecer muitas ferramentas. É saber quando usar, quando evitar, como operar, como proteger, como recuperar e como justificar tecnicamente cada decisão em produção.